Faculty of Arts and Sciences

The Department of Computer Science

CIS1703
Programming 2

Level 4

Coursework 2 (Group Project)
2025/2026

Student Names: Aaron Rielly (26583160), Benjamin McManus (26611830), Daniel Caveney (26488426), Lewis Jones (25353179), Eesaa Burtwistle (25295748)

Team: 3

Table of Contents

Introduction

This is the document and report of our Coursework 2 for the programming 2 course, the group project brief that has been selected is the “smart stock” intelligent inventory management system, the theme is specific to one thing, so we get to choose what to base it on.

The main goal of this project is to create a smart inventory management system that not only count items; they predict needs and manage waste; we are building a system for a high-tech retail hub for the client based on the criteria given. For the 3 components required of the program, that being the Domain Entities, Functional Requirements and the HCI requirements.

Domain Entities: the requirements involve a base class and two subclasses, with the base class being a product with an (ID, Name, Price, and Quantity). The subclass 1 is a PerishableProduct with a (Adds: Expiry Date, and Storage Temp), and subclass 2 is an ElectronicProduct with a (Adds: Warranty Period, and Power Usage).

Functional Requirements:
1. Stock Management: Add, Edit, and Remove items.
2. Smart Alerts: Low Stock Warning: Automatically flag items that fall below a defined threshold (e.g., < 5 units). Expiry Detection: Upon startup, the system should generate a report of all items expiring within the next 7 days.
3. Value Calculation: A function to calculate the total financial value of the current stock, applying a depreciation formula to the ElectronicProducts.
4. Transaction History: Log every time an item is added or removed to a separate log file (audit trail).

HCI Requirement: the requirements involve creating a “Dashboard” like interface view for the user, whether this is CLI or GUI, that helps visually summarise the health of the inventory, displaying important information such as Total Items, Low Stock Alerts, Total Value, all within a single glance.

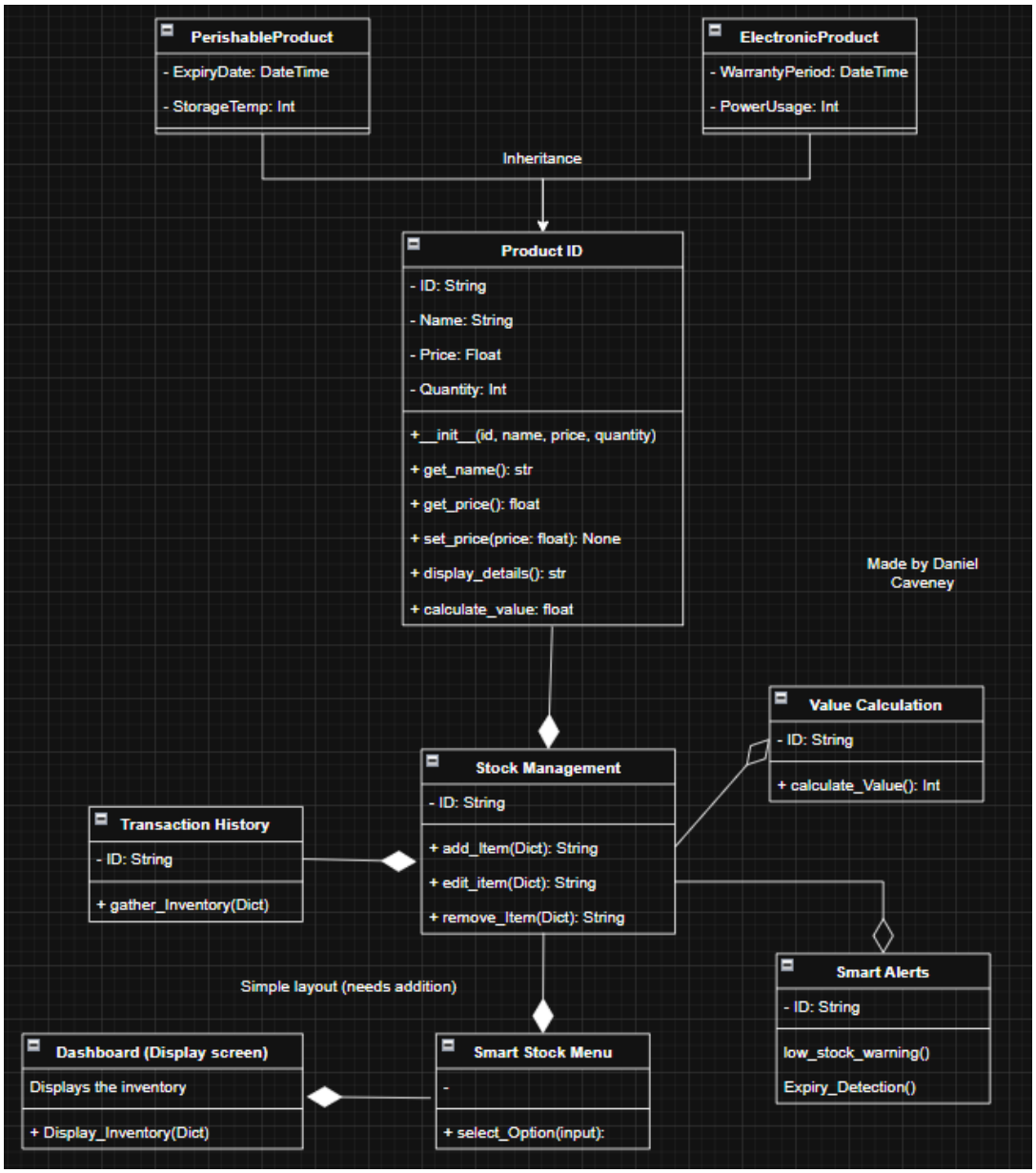
Finally, the brief mentions the main marking focus areas, meaning we need to prioritise making the best version out of these areas to gain the most marks.

Marking Focus Areas: Object-Oriented Architecture, Robust Data Persistence, Smart Logic & Defensive Coding, and HCI & Usability.

Written by Daniel Caveney

Architectural design

This area is to focus upon the UML design; this is to be made before the implementation of code and the dashboard GUI/CLI. This will include the process of the UML design and the final design that will be used as a base for the actual code.

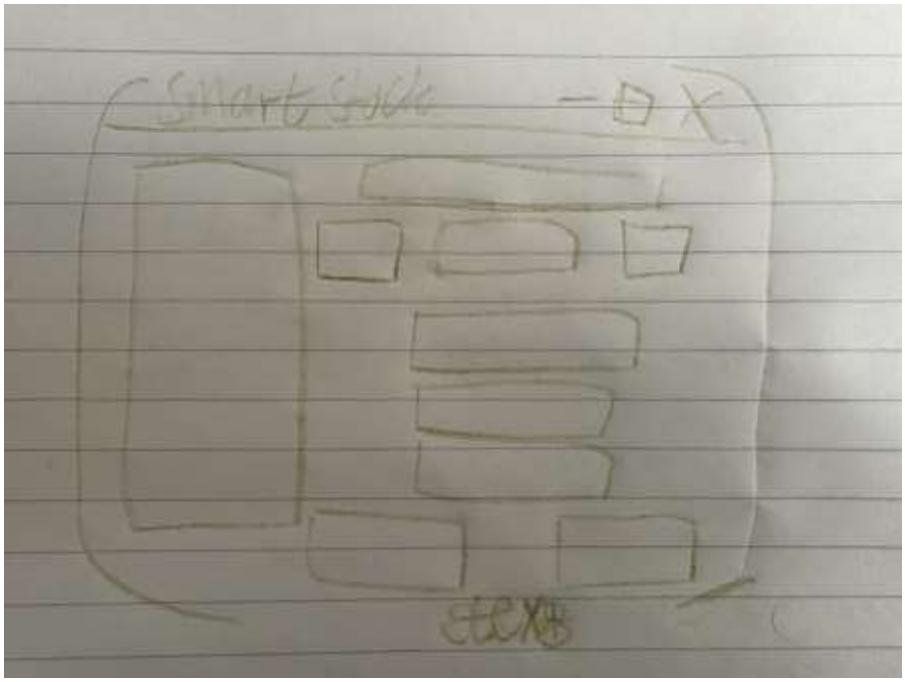


This is the first design for the UML, while there are some clear things that need to be added to the design, this allows us to get a basis on what the brief is expecting at a minimum, as it contains the domain entities that the brief required, that being the Product ID class and then the ElectronicProduct and PerishableProduct subclass that inherits from the Product ID class. It also provides the other areas that was stated to be needed in the program by the brief, this allows us to get an understanding of how the program as a whole will work, such as having the dashboard (GUI/CLI) being connected to a smart stock menu that has strong ownership over the stock management area, which connects all the parts into the menu.

This is likely the final design as this was designed and updated to accommodate for the changes to the program that the group believes was fitting, however still this doesn't mean that the program will need to follow this one to one, it is just extremely useful to have.

Written by Daniel Caveney

Wireframe for lo2



This is the wireframe that is designed for the Smart stock, with there being a clear title present, then on the left a large and long dashboard for the viewing of all the stock present inside, on the right side there is all the buttons and the entry label so the user can input what they need (and must correctly input something for the button to work to adhere to defensive coding) and depending on the button, such as "add" or "remove", the specified command will occur with text at the very bottom stating what happened for the user and any errors that might have happened if the user made a mistake.

This doesn't have to be followed one to one, however it should be kept in mind when making the finalised program, so I made this so everyone on the team can have an idea of what the program should look like for the user.

Written by Dan Caveney

Implementation of a HCI Compliant Interface

1. Visibility of System Status

The interface maintains strong visibility of system status by continuously informing the user about ongoing operations and system conditions. This is primarily achieved through the status_label which updates dynamically to reflect actions such as adding, editing, deleting or saving stock items. In addition, the update_dashboard function provides real time summaries of the total inventory quantity and overall stock value. The smart alerts button enhances visibility by informing the user of items that are currently low in stock by using a visual cue (red text) when attention is required. Together, these features ensure that users are never uncertain about the systems current state. Additionally, the system enhances visibility by detecting perishable items nearing expire and notifying the user both through the smart alerts system and automatically on startup. The dashboard summary windows further improves system transparency by presenting key metrics and system health indicators by using colour based feedback.

2. Match Between System and the Real World

The system closely mirrors real world inventory management concepts making it intuitive for users. Terminology such as ‘Add Stock’, ‘Remove Stock’, ‘Edit Stock’ and ‘Transaction History’ reflect language commonly used in retail or warehouse environments. Data fields such as price, quantity, expiry date and warranty period are also realistic and meaningful. Furthermore, the use of the pound symbol (£) for currency reinforces familiarity for users within the UK. This alignment reduces cognitive effort required to understand and operate the system.

3. User Control and Freedom

The interface provides users with a degree of control over their actions, particularly through the use of secondary windows (TopLevel) for adding and editing stock which can be closed at any time to cancel the operation. Also, the deletion process includes a confirmation dialog (askyesno) which prevents accidental removal of data. However, while these features support basic control the system lack an undo feature for reversing completed action such as deletions or edits. As a result, user freedom is only partially supported as mistakes cannot be easily corrected after confirmation.

4. Consistency and Standards

Consistency is maintained well throughout the interface, both visually and functionally. All buttons share a uniform style defined by a common configuration (btn_style) ensuring a cohesive appearance. The layout and structure of input forms in both the ‘Add Stock’ and ‘Edit Stock’ windows are highly similar which helps users transfer knowledge from one task to another. Standard GUI conventions such as buttons for actions and dialog boxes for warning or confirmations are followed consistently. This adherence to established patterns improves usability by reducing the learning curve.

5. Error Prevention

The program uses multiple layers of validation to prevent user error before they occur. Input fields are checked to ensure they are not left empty and data types are strictly enforced (prices must be a float and quantities must be an integer). Date inputs are validated against a specific format (DD/MM/YY) and incorrect entries trigger immediate warning messages. The use of radio buttons for selecting product types also prevents invalid category input. By catching errors early, the system minimises the likelihood of incorrect or corrupted data being entered. Also, validation is context sensitive which ensures that perishable items require a correctly formatted date, while electronic items require a numeric warranty value e.g. 12 or 24.

6. Recognition Rather Than Recall

The interface is designed to reduce the users memory load by emphasising recognition over recall. The inventory list box displays all current stock items in a clear, readable format which allows users to more easily identify and select items without needing to remember details. When editing an item, existing values are prefilled into the input fields which allows users to modify data without re-entering it from memory. Labels next to each input field further clarify what information is required and therefore make the interaction straightforward and user friendly.

7. Flexibility and Efficiency of Use

The system provides a level of efficiency for typical users by offering direct access to core functions through clearly labelled buttons such as ‘Add Stock’, ‘Edit Stock’ and ‘Dashboard View’. The dashboard itself improves efficiency by summarising key metrics in a single view, reducing the need for manual calculations. However, the interface does not include advanced features such as keyboard shortcuts or a search function which would enhance usability for more experienced users. The dedicated dashboard summary window allows users to quickly access aggregated analytics such as total products, stock levels and percentage of low stock levels which improves efficiency in decision making.

8. Aesthetic and Minimalist Design

The GUI follows a relatively clean layout and minimalist design which presents only essential information and controls. The layout separates the inventory list from the action buttons, creating a clear distinction between data display and interaction. The use of simple fonts and minimal decorative elements ensures that the interface remains uncluttered and easy to navigate. However, the vertical stacking of buttons could be refined through grouping or visual segmentation to improve readability and organisation. Overall, the design is simple and functional. The use of colour coding within the dashboard (red for critical stock levels and green for healthy stock levels) improves visual clarity without adding unnecessary complexity.

9. Help Users Recognise, Diagnose and Recover from Errors

Error handling in the system is clear and informative which allows users to understand and correct issues easily. When invalid input is detected, the program displays specific and descriptive warning messages (e.g. price must be a float, or quantity must be an integer). These messages directly identify the problem and

guide the user toward the correct input format. Additionally, confirmation dialogs help prevent accidental actions. While recovery options such as undo are limited, the system effectively supports error recognition and diagnosis.

10. Help and Documentation

The system provides minimal help or documentation within the interface. Users rely mostly on intuitive design, clear labels and validation messages to understand how to use the application. While this may be sufficient for simple tasks, there is no dedicated help menu, user guide or tooltip system to assist new users or explain more complex features such as transaction history or dashboard metrics. As a result, this heuristic is only met to a certain degree and is an area for potential improvement.

Written by Lewis Jones

Testing/ QA Tests

Task 1: Adding new items to the SmartStock

Test number	Test For	Type of test	Expected result	Actual result
1	Add Stock	Normal	When inputting all the data correctly, no errors will occur	PASS
2	Add Stock	Normal	When inputting a new item into the inventory, no errors occur	PASS
3	Add Stock	Normal	When inputting a new item, inputting an integer works even though the price variable is a float.	PASS
4	Add Stock	Normal	Clicking the “main” start button will cause the program to run fully and as intended, allowing us to click on the add stock button with no issues	PASS
5	Add Stock	Normal	When inputting a expiry date or warranty, inputting the date time format correctly will result in it working correctly	PASS
6	Add Stock	Erroneous	When clicking the “save product” button without inputting anything, an error message pops up	PASS
7	Add Stock	Erroneous	When clicking the “save product” button with one item missing an error message pops up	PASS
8	Add Stock	Erroneous	When clicking the “save product” button with price not being the correct data type, a specific error message pops up saying so	PASS
9	Add Stock	Erroneous	When inputting a expiry date or warranty, inputting the date time format incorrectly will result in an error	PASS
10	Add Stock	Extreme	When inputting a new item, inputting “£4”, this shouldn’t work because it isn’t a float or int	PASS
11	Add Stock	Extreme	When inputting a new item, inputting “0” should work because it is formatted as an int	PASS
12 and 13	Add stock	Extreme	When inputting a expiry date or warranty, inputting the 29/02/26 will not pass and result in an error because it isn’t a leap year, but 29/02/24 does pass	PASS

Tested by Daniel Caveney

Task 2: Editing items to the SmartStock

Test number	Test For	Type of test	Expected result	Actual result
1	Edit Stock	Normal	When editing one piece of data correctly, no errors will occur	PASS
2	Edit Stock	Normal	When editing all parts of the item correctly, no errors will occur	PASS
3	Edit Stock	Normal	When the user presses an item and presses the edit stock button, everything works as intended	PASS
4	Edit Stock	Erroneous	If an item hasn’t been selected to edit and the user presses the edit button, then an error message appears stating nothing has been selected	PASS

5	Edit Stock	Erroneous	If all the data from the item is removed and the user presses save, the program shouldn't accept this	The program accepted it; this means there should be some additional code (fail)
6	Edit Stock	Erroneous (Second test)	If all the data from the item is removed and the user presses save, the program shouldn't accept this.	The expected result occurred. The fail is now successful thanks to the added defence code
	Edit Stock	Erroneous	Editing the expiry date/ warranty and changing the format to be incorrect will result in an error message	PASS
7	Edit Stock	Extreme	Changing the data in ways it shouldn't, will be accepted but can also be edited right after. (this was done to give the edit button a lot of versatility)	PASS
8	Edit Stock	Erroneous	Changing the data in ways it shouldn't, won't be accepted and display an error message	The expected result occurred. This was changed to be more appropriate
9	Edit Stock	Extreme	Changing an items quantity by inputting "0" should work because it is formatted as an int	PASS
10	Edit Stock	Normal	Editing the stock levels of an electronic item already saved in the database will change the stock level without an error.	The expected result did not occur. An error with the date handling of the warranty of an electronic item was spotted and fixed by Eesaa after being caught by Aaron. The expected result does now occur. (PASS)

Test 1-9 tested by Daniel Caveney, Test 10 tested by Aaron Rielly

Task 3: Removing items to the SmartStock

Test number	Test For	Type of test	Expected result	Actual result
1	Remove Stock	Normal	When the user selects an item and clicks the remove stock button, it gives the user a second window to choose yes or no	PASS
2	Remove Stock	Normal	If the user selects "yes" in the second window, then the stock is promptly removed	PASS
3	Remove Stock	Normal	If the user selects "No" in the second window, then the stock remains and isn't removed	PASS
4	Remove Stock	Erroneous	If an item hasn't been selected to remove and the user presses the remove button, then an error message appears stating nothing has been selected	PASS

Tested by Daniel Caveney

Task 4: Smart Alerts for the SmartStock

Test number	Test For	Type of test	Expected result	Actual result
1	Smart Alerts	Normal	When a stock is added above the smart alerts threshold no smart alert will be notified.	PASS
2	Smart Alerts	Normal	When a stock is added below the smart alerts' threshold, a smart alert will be notified to the user. An error message will appear displaying what item needs to be restocked	PASS
3	Smart Alerts	Normal	When a stock is below the smart alert threshold and then edited to be above, the smart alert will fade	PASS
4	Smart Alerts	Extreme	When a stock's quantity is added as 5, then there shouldn't be a smart alert as it has to be less than 5.	PASS
5	Smart Alerts	Normal	When perishable stock added to the system has an expiry date of greater than 7 days away no smart alert is generated.	PASS
6	Smart Alerts	Normal	When perishable stock added to the system has an expiry date of less than 7 days away a smart alert is generated.	PASS
7	Smart Alerts	Normal	When perishable stock added to the system has an expiry date of less than 7 days the smart alert button on the dashboard is changed to reflect the extra smart alert. e.g. Smart Alerts (1) for one smart alert.	The expected result did not occur. The Smart Alerts button on the dashboard currently only recognises low stock items. This issue has now been corrected by Aaron, and the expected result does now occur (PASS)
8	Smart Alerts	Normal	On start-up of the system when there is perishable withs an expiry date of less than 7 an alert will automatically generate.	PASS
9	Smart Alerts	Extreme	When a perishable item's expiry date is exactly 7 days away there shouldn't be a smart alert as it must be less than 7.	PASS

Tests 1-4 tested by Daniel Caveney, Tests 5-9 tested by Aaron Rielly

Task 5: Value Calculations for the SmartStock

Test number	Test For	Type of test	Expected result	Actual result
1	Value Calculations	Normal	Once an item has been added, clicking on the value calculations it will automatically calculate the total stock value, by doing Price x Quantity. Testing with just one item	PASS
2	Value Calculations	Normal	Clicking on the value calculations it will automatically calculate the total stock value, by doing Price x Quantity. Testing with just two items	PASS
3	Value Calculations	Extreme	Clicking on the value calculations it will automatically calculate the total stock value, by doing Price x Quantity. Testing with just three items where one has no stock	PASS
4	Value Calculations	Erroneous	Clicking on the value calculations when there aren't any items in the inventory will show a warning message saying "No stock available"	PASS

Tested by Daniel Caveney

Task 6: Transaction history for the SmartStock

Test number	Test For	Type of test	Expected result	Actual result
1	Transaction History	Normal	When the inventory has items inputted into it, a window will open showing all of the adds, edits and removals made by the user.	PASS
2	Transaction History	Normal	When the user edits a value of a variable, then the open the transaction history, the change will be there	PASS
3	Transaction History	Normal	When the program is first ran, clicking on the transaction history will display all the past transaction even when the program has been terminated.	PASS

Tested by Daniel Caveney

Task 7: Save SmartStock data

Test number	Test For	Type of test	Expected result	Actual result
1	Save	Normal	Clicking the save button where the list is empty will still result in the Status label saying "Inventory saved to JSON"	PASS
2	Save	Normal	Clicking the save button where the list has multiple items will result in the status label saying "Inventory saved to JSON"	PASS
3	Save	Normal	Opening the program after it being previously terminated where the inventory list items saved will remain in the list	It saves the data into the JSON file; however, it doesn't load upon the program being loaded again after termination (FAIL)
4	Save	Normal	Opening the program after it being previously terminated where the inventory list items saved will remain in the list. Attempt 2	It saves the data into the JSON file and when the user terminates and reopens, the program will open with all the saved information inside the program. (PASS)

Tested by Daniel Caveney

Task 8: Dashboard View for SmartStock

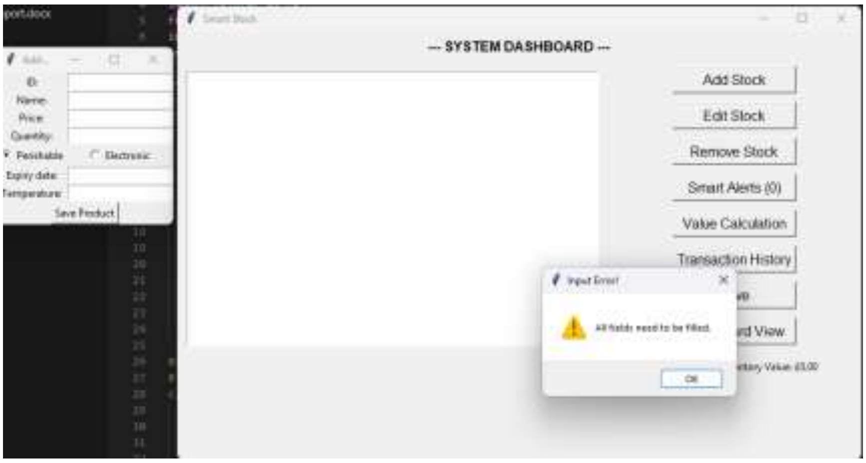
Test number	Test For	Type of test	Expected result	Actual result
1	Dashboard view	Normal	When the status label is shown and the dashboard button is pressed, the dashboard view will be shown	PASS
2	Dashboard View	Erroneous	When the dashboard button is pressed with no data inside the inventory list, a warning message will appear	PASS
3	Dashboard View	Normal	When the dashboard button is pressed and there are stocks inside, if all have the correct number of stocks then the dashboard will show in green text all stocks are okay	PASS
4	Dashboard	Normal	When the dashboard button is pressed and there are stocks inside, if any of the stocks inside are below the stock threshold (<5) then the dashboard will show red text stating there is a stock that needs replacing, as well as listing which one(s).	PASS

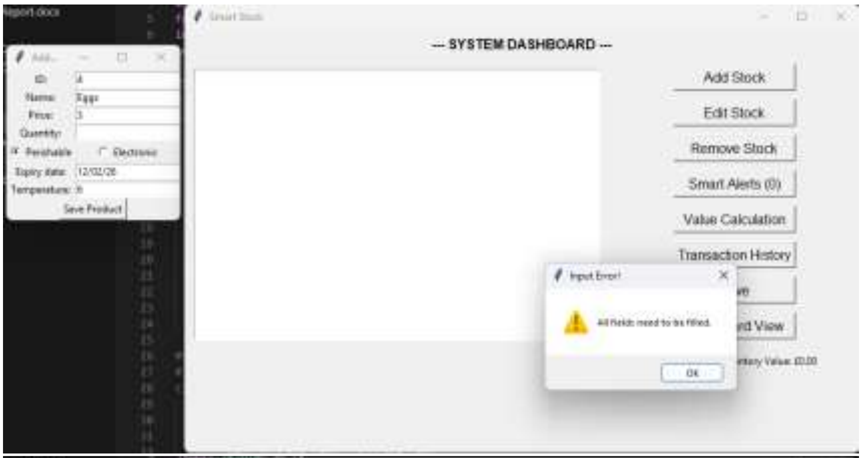
Tested by Daniel Caveney

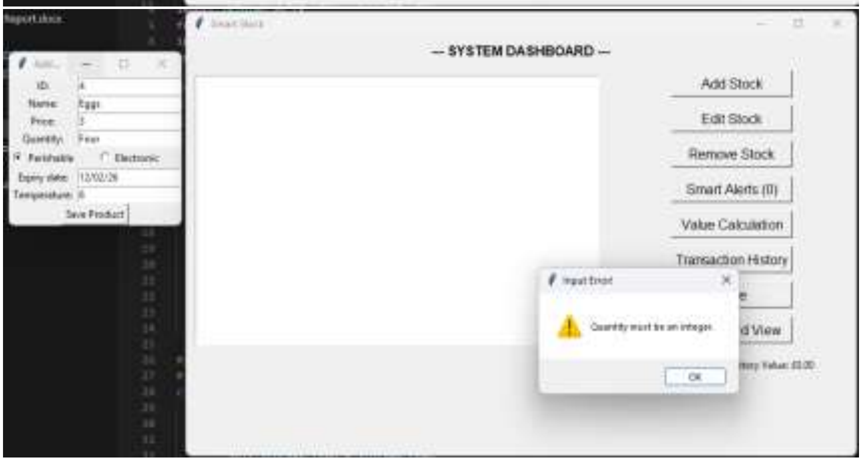
Erroneous case evidence

Case 1: Adding new items to the SmartStock

6	Add Stock	Erroneous	When clicking the "save product" button without inputting anything, an error message pops up	PASS
7	Add Stock	Erroneous	When clicking the "save product" button with one item missing an error message pops up	PASS
8	Add Stock	Erroneous	When clicking the "save product" button with price not being the correct data type, a specific error message pops up saying so	PASS

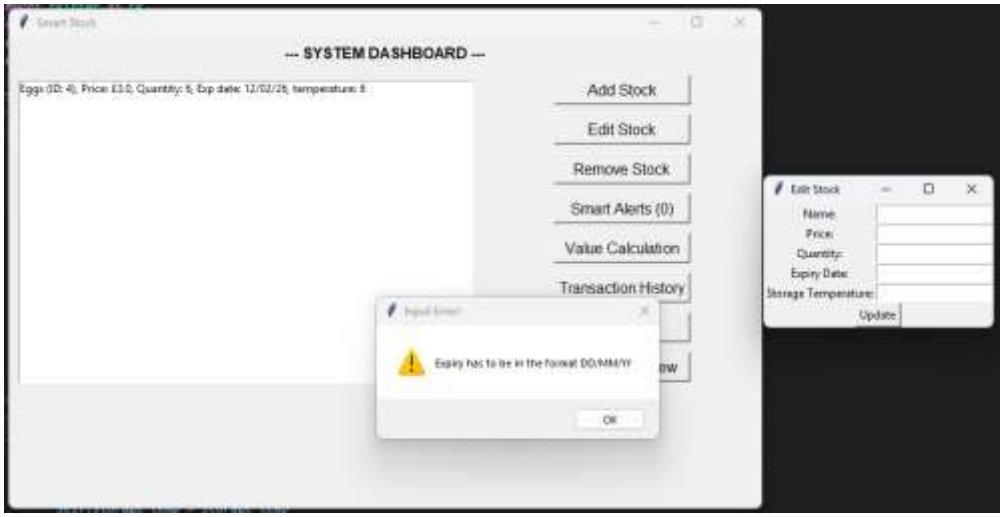






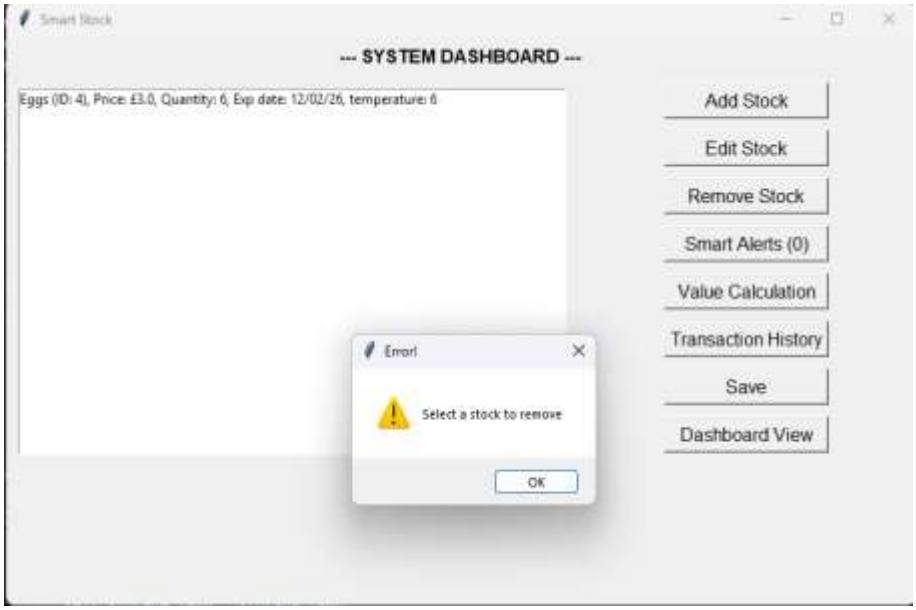
Case 2: Editing items in the SmartStock

4	Edit Stock	Erroneous	If an item hasn't been selected to edit and the user presses the edit button, then an error message appears stating nothing has been selected	PASS
6	Edit Stock	Erroneous (Second test)	If all the data from the item is removed and the user presses save, the program shouldn't accept this.	The expected result occurred. The fail is now successful thanks to the added defence code



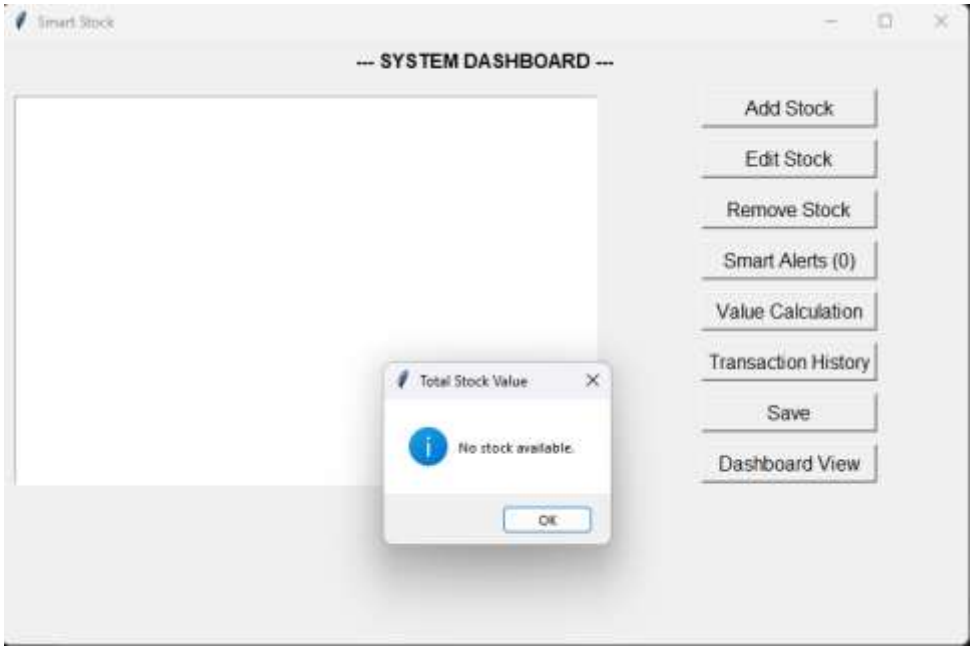
Case 3: Removing items from the SmartStock

4	Remove Stock	Erroneous	If an item hasn't been selected to remove and the user presses the remove button, then an error message appears stating nothing has been selected	PASS



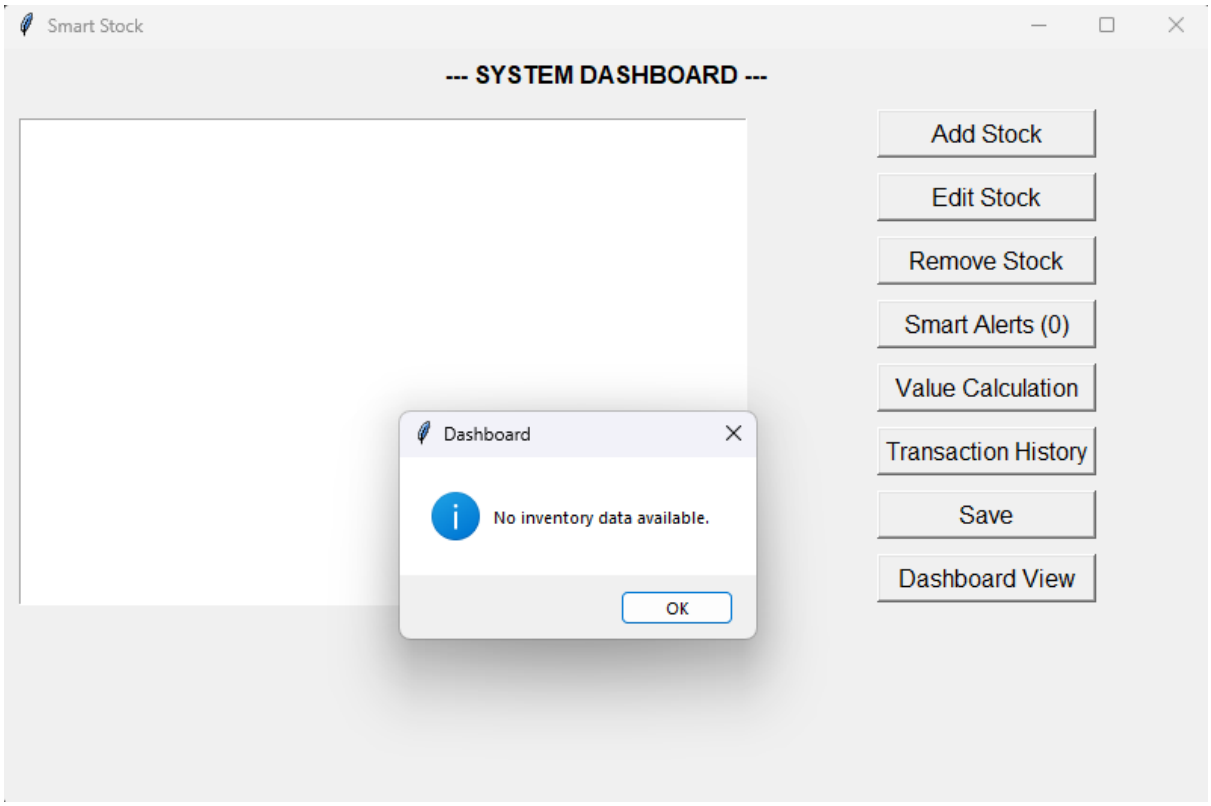
Case 4: Value Calculations for the SmartStock

4	Value Calculations	Erroneous	Clicking on the value calculations when there aren't any items in the inventory will show a warning message saying "No stock available"	PASS

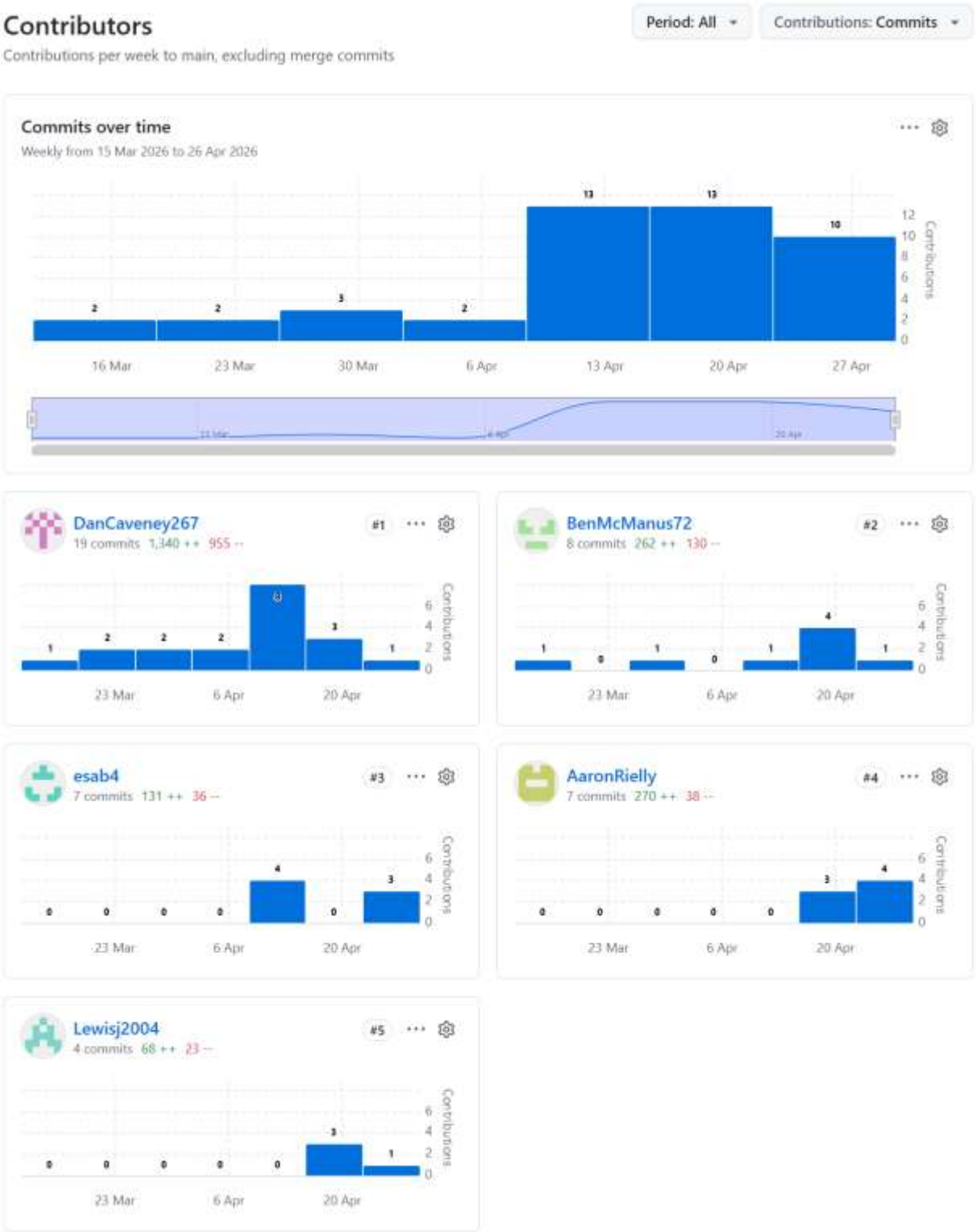


Case 5: Dashboard View for SmartStock

2	Dashboard View	Erroneous	When the dashboard button is pressed with no data inside the inventory list, a warning message will appear	PASS



GitHub Contributors Graph



Meeting Minutes

Meeting #	Date & Time	Attendees	Key Decisions	Action Items (Who + What)
1	26/03/26 18:20	Aaron Rielly, Ben McManus, Dan Caveney	- Scenario A: Smart Stock system chosen. - Roles Decided	- Ben (Group Lead) - Dan (QA Lead) - Aaron (Developer) - Dan to design initial UML - Ben to set up GitHub
2	15/04/26 17:30	Aaron Rielly, Ben McManus, Dan Caveney, Lewis Jones	- Lewis to take on role as UI/UX Developer	- Lewis to upload core UI/UX code to GitHub repo - Ben and Aaron to continue coding SmartStock System
3	20/04/26 19:15	Aaron Rielly, Ben McManus, Dan Caveney, Eesaa Burtwistle, Lewis Jones	- Eesaa to take on Developer role with Aaron	- Aaron to create README file - Dan to finalise testing for the SmartStock system - Aaron, Ben, Eesaa & Lewis to complete system - Ben to compile presentation

Conclusion

On reflection, the group worked very well. Everyone was fairly punctual with their work and produced a high standard of code. however, if we were to do things differently we would split the work evenly to begin with and organise it so everyone knows everything that they will have to do from day 1, instead of it forming organically. This is so then the workload can get spread more evenly, and there can be higher levels of accountability. This would be the case because then we would know in advance who has to work on what, meaning that we can ask them quicker. This would also solve another problem that we came across, as later on in the project there were still parts of the code that was left unclaimed. All work got done this time, but with a different group this may not be the case, and due to the lower levels of accountability this time around it would have been hard to designate work to one person.

In conclusion, it is fair to say that the group worked effectively together. Everyone could be trusted to pull their own weight, relieving stress from all of the other team members. Everyone picked work that made up around 20% of the project, meaning the work was evenly distributed. People where responsive when issues came up, and when meetings where planned everyone knew what to do. People successfully uploaded to another branch other than main for people to test before pushing to main, and overall the whole group had high engagement, meaning that working together was much more fun and effective than it could have been with another group.

Written by Ben McManus